

## Object-Oriented Programming

### Study Plan

| Level        | Topics                                   | Practice Ideas                                   |
|--------------|------------------------------------------|--------------------------------------------------|
| Beginner     | What is OOP, Class, Object, Constructor  | Create simple Car, Person classes                |
| Intermediate | Encapsulation, Inheritance, Polymorphism | Create a “Bank System” or “Animal” hierarchy     |
| Advanced     | Abstract class, Interface, Generics      | Build small project (Employee Management)        |
| Testing Use  | Page Object Model (POM)                  | Create sample Selenium/Playwright test using OOP |

### What is OOP?

**OOP (Object-Oriented Programming)** is a programming style where we design software using **objects** - real-world entities like *Car*, *Employee*, *Student*, *Account*, etc.

Each object has:

- **Properties (data)** => e.g., name, color, price
- **Methods (actions)** => e.g., start(), stop(), calculateSalary()

In OOP, we model the program like how we see things in real life.

### Why Do We Use OOP?

| Benefit       | Description                                              |
|---------------|----------------------------------------------------------|
| Reusability   | Write code once, reuse it in other places.               |
| Modularity    | Break program into smaller classes for easy maintenance. |
| Abstraction   | Hide complex logic, show only needed details.            |
| Encapsulation | Protect data from direct modification.                   |
| Inheritance   | Reuse and extend existing code.                          |
| Polymorphism  | Perform the same operation in different ways.            |
| Easy Testing  | Classes can be tested independently (unit testing).      |

In short: **OOP = organized + reusable + testable code.**

## Pillars of OOP

| Concept              | Meaning                                             | Example                            |
|----------------------|-----------------------------------------------------|------------------------------------|
| <b>Encapsulation</b> | Binding data + methods together; hiding details     | Private variables, getters/setters |
| <b>Abstraction</b>   | Showing only essentials, hiding complex parts       | Abstract classes, interfaces       |
| <b>Inheritance</b>   | One class derives from another                      | class Student extends Person       |
| <b>Polymorphism</b>  | One method behaves differently in different classes | Method overriding / overloading    |

## Key OOP Concepts

| Concept              | Definition                                                                                                    |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Class</b>         | A blueprint for creating objects (like a template).                                                           |
| <b>Object</b>        | An instance of a class (a real usable thing).                                                                 |
| <b>Encapsulation</b> | Hiding data inside the class using private or protected keywords.                                             |
| <b>Inheritance</b>   | Reusing properties and methods from one class to another using extends.                                       |
| <b>Polymorphism</b>  | Performing a single action in different ways (method overriding).                                             |
| <b>Abstraction</b>   | Hiding complex implementation details and showing only the necessary parts using abstract class or interface. |

## Class and Object

### Definition:

A **class** is like a blueprint, and an **object** is a real thing built from it.

### Example Code

```
class Car {  
  brand: string;  
  model: string;  
  
  constructor(brand: string, model: string) {  
    this.brand = brand;  
    this.model = model;  
  }  
  
  start() {  
    console.log(`${this.brand} ${this.model} is starting...`);  
  }  
}  
  
const car1 = new Car("Toyota", "Innova");  
car1.start();
```

### Explanation:

- `class Car` defines a car blueprint.
- `constructor()` initializes `brand` and `model`.
- `car1` is an object created using `new`.
- `start()` prints a message.

### Output

*// Toyota Innova is starting...*

## Encapsulation

### Definition:

Encapsulation means **protecting data** inside a class and allowing controlled access using methods (getters/setters).

### Example Code

```
class Account {  
  private balance: number;  
  
  constructor(initialBalance: number) {  
    this.balance = initialBalance;  
  }  
  
  deposit(amount: number) {  
    this.balance += amount;  
  }  
  
  getBalance(): number {  
    return this.balance;  
  }  
}  
  
const acc = new Account(1000);  
acc.deposit(500);  
console.log("Current Balance:", acc.getBalance());
```

### Explanation:

- `private balance` hides the data from direct access.
- `deposit()` and `getBalance()` provide controlled access.
- You cannot access `acc.balance` directly — only through methods.

### Output

*// Current Balance: 1500*

## Inheritance

### Definition:

**Inheritance** allows one class to use and extend another class's properties and methods using `extends`.

### Example Code

```
class Animal {  
  eat() {  
    console.log("Eating...");  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log("Barking...");  
  }  
}  
  
const dog = new Dog();  
dog.eat();  
dog.bark();
```

### Explanation:

- `Dog extends Animal` → Dog inherits all features from Animal.
- Dog can use both `eat()` (from parent) and `bark()` (its own method).

### Output

*// Eating...*

*// Barking...*

## Polymorphism

### Definition:

- **Polymorphism** means “many forms.”
- The same method name can behave differently in different classes.

### Example Code

```
class Shape {
  area(): number {
    return 0;
  }
}

class Circle extends Shape {
  constructor(private radius: number) {
    super();
  }
  area(): number {
    return 3.14 * this.radius * this.radius;
  }
}

class Rectangle extends Shape {
  constructor(private width: number, private height: number) {
    super();
  }
  area(): number {
    return this.width * this.height;
  }
}

const shapes: Shape[] = [new Circle(5), new Rectangle(4, 6)];
shapes.forEach(s => console.log("Area:", s.area()));
```

### Explanation:

- Shape is a parent class with a method `area()`.
- Circle and Rectangle override `area()` differently.
- The same method call gives different results.

## Output

*// Area: 78.5*

*// Area: 24*

## Abstraction

### Definition:

Abstraction **hides unnecessary details and only shows essential features using abstract class or interface.**

### Example Code

```
interface IEmployee {
    empId: number;
    name: string;
    getSalary(): number;
}

class Developer implements IEmployee {
    constructor(public empId: number, public name: string, private salary: number) {}

    getSalary(): number {
        return this.salary;
    }
}

const dev = new Developer(101, "Ravi", 60000);
console.log("Employee Name:", dev.name);
console.log("Salary:", dev.getSalary());
```

## Explanation:

- `abstract class` → can't create objects directly.
- `start()` → abstract method must be implemented in subclass.
- Hides internal complexity.

## Output

*// Bike Started*

*// Vehicle Stopped*

## Interface

### Definition:

- An interface **defines a structure or contract that a class must follow.**
- It doesn't contain logic - only rules.

### Example Code

```
interface IEmployee {
    empId: number;
    name: string;
    getSalary(): number;
}

class Developer implements IEmployee {
    constructor(public empId: number, public name: string, private salary: number) {}

    getSalary(): number {
        return this.salary;
    }
}

const dev = new Developer(101, "Ravi", 60000);
console.log("Employee Name:", dev.name);
console.log("Salary:", dev.getSalary());
```



## Explanation:

- interface `IEmployee` defines what every employee must have.
- Developer class implements it and provides real logic.

## Output

*// Employee Name: Ravi*

*// Salary 60000*

## Example for Testers (POM)

### Definition

In automation testing, we use OOP to create reusable **Page Classes** for each web page – called **POM (Page Object Model)**.

```
class LoginPage {
    open() {
        console.log("Opening Login Page...");
    }

    login(username: string, password: string) {
        console.log(`Logging in with username: ${username} and password: ${password}`);
    }
}

class LoginTest extends LoginPage {
    runTest() {
        this.open();
        this.login("admin", "admin123");
        console.log("✅ Login Test Passed!");
    }
}

const test = new LoginTest();
test.runTest();
```

### **Explanation:**

- LoginPage → page class with methods.
- LoginTest → inherits and performs the test steps.
- This structure is widely used in **Playwright / Selenium automation**.

### **Output**

*// Opening Login Page...*

*Logging in with username: admin and password: admin123*

*Login Test Passed!*

### **Practice Tasks**

1. Create a class Student with name, age, marks
2. Create Teacher extending Person
3. Implement IShape interface for Circle, Rectangle
4. Create a LoginTest using LoginPage class
5. Try to apply Encapsulation for BankAccount